

HDOC Day-34

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D arrays and verify the left distributive property:
 $A*(B+C) = (A*B) + (A*C)$. The dimensions of input matrices are given as follows: A (m x n), B (n x p), C (n x p).
2. Read three variable-length 2D arrays and verify the right distributive property:
 $(A+B)*C = (A*C) + (B*C)$. The dimensions of input matrices are given as follows: A (m x n), B (m x n), C (n x p)
3. Exit

Algorithm:

1. Start.
2. Display the menu:
 - Verify left distributive property.
 - Verify right distributive property.
 - Exit.
3. Read the user's choice.
4. If choice is 1, call leftDistributive().
5. If choice is 2, call rightDistributive().
6. If choice is 3, terminate the program.
7. Otherwise, display **Invalid choice**.
8. Repeat the menu until the user selects Exit.
9. Stop.

```
#include <stdio.h>
```

```
/* Function to read a matrix */
```

```
void readMatrix(int rows, int cols, int matrix[rows][cols])
```

```
{
```

```
    int i, j;
```

```
    for (i = 0; i < rows; i++)  
    {  
        for (j = 0; j < cols; j++)  
        {  
            scanf("%d", &matrix[i][j]);  
        }  
    }  
}
```

/* Function to display a matrix */

```
void displayMatrix(int rows, int cols, int matrix[rows][cols])  
{  
    int i, j;  
  
    for (i = 0; i < rows; i++)  
    {  
        for (j = 0; j < cols; j++)  
        {  
            printf("%d\t", matrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

/* Function to add two matrices */

```
void addMatrix(int rows, int cols,  
               int A[rows][cols],
```

```

        int B[rows][cols],
        int R[rows][cols])
{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            R[i][j] = A[i][j] + B[i][j];
        }
    }
}

```

/* Function to multiply two matrices */

```

void multiplyMatrix(int r1, int c1,
                    int A[r1][c1],
                    int r2, int c2,
                    int B[r2][c2],
                    int R[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            R[i][j] = 0;

```

```

        for (k = 0; k < c1; k++)
        {
            R[i][j] += A[i][k] * B[k][j];
        }
    }
}
}

```

/* Function to compare two matrices */

```

int compareMatrix(int rows, int cols,
                  int A[rows][cols],
                  int B[rows][cols])

```

```

{
    int i, j;

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            if (A[i][j] != B[i][j])
                return 0;
        }
    }
}

```

```

    return 1;
}

```

```

/* Left distributive property:  $A(B+C) = AB + AC$  */
void leftDistributive()
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    int A[m][n];
    int B[n][p];
    int C[n][p];

    int BplusC[n][p];
    int left[m][p];

    int AB[m][p];
    int AC[m][p];
    int right[m][p];

    printf("\nEnter elements of Matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("\nEnter elements of Matrix B (%d x %d):\n", n, p);
    readMatrix(n, p, B);

    printf("\nEnter elements of Matrix C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

```

```

/* B + C */
addMatrix(n, p, B, C, BplusC);

/* A(B + C) */
multiplyMatrix(m, n, A, n, p, BplusC, left);

/* AB */
multiplyMatrix(m, n, A, n, p, B, AB);

/* AC */
multiplyMatrix(m, n, A, n, p, C, AC);

/* AB + AC */
addMatrix(m, p, AB, AC, right);

printf("\nLeft Hand Side: A(B + C)\n");
displayMatrix(m, p, left);

printf("\nRight Hand Side: AB + AC\n");
displayMatrix(m, p, right);

if (compareMatrix(m, p, left, right))
    printf("\nLeft distributive property is VERIFIED.\n");
else
    printf("\nLeft distributive property is NOT VERIFIED.\n");
}

/* Right distributive property: (A+B)C = AC + BC */

```

```
void rightDistributive()
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    int A[m][n];
    int B[m][n];
    int C[n][p];

    int AplusB[m][n];
    int left[m][p];

    int AC[m][p];
    int BC[m][p];
    int right[m][p];

    printf("\nEnter elements of Matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("\nEnter elements of Matrix B (%d x %d):\n", m, n);
    readMatrix(m, n, B);

    printf("\nEnter elements of Matrix C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    /* A + B */
```

```

addMatrix(m, n, A, B, AplusB);

/* (A + B)C */
multiplyMatrix(m, n, AplusB, n, p, C, left);

/* AC */
multiplyMatrix(m, n, A, n, p, C, AC);

/* BC */
multiplyMatrix(m, n, B, n, p, C, BC);

/* AC + BC */
addMatrix(m, p, AC, BC, right);

printf("\nLeft Hand Side: (A + B)C\n");
displayMatrix(m, p, left);

printf("\nRight Hand Side: AC + BC\n");
displayMatrix(m, p, right);

if (compareMatrix(m, p, left, right))
    printf("\nRight distributive property is VERIFIED.\n");
else
    printf("\nRight distributive property is NOT VERIFIED.\n");
}

int main()
{

```

```
int choice;

do
{
    printf("\n===== MATRIX DISTRIBUTIVE PROPERTY =====\n");
    printf("1. Verify Left Distributive Property\n");
    printf("2. Verify Right Distributive Property\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            leftDistributive();
            break;

        case 2:
            rightDistributive();
            break;

        case 3:
            printf("\nProgram terminated.\n");
            break;

        default:
            printf("\nInvalid choice! Please enter 1, 2 or 3.\n");
    }
}
```

```

    } while (choice != 3);

    return 0;
}

```

Test Case Table:

Test Case	Choice	Dimensions (m, n,p)	Expected Result
1	1	2 2 2	Left distributive property is verified.
2	2	2 2 2	Right distributive property is verified.
3	1	2 3 2	Left distributive property is verified
4	2	2 3 2	Right distributive property is verified.
5	3	-	Program terminates
6	4	-	Invalid choice

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:

```
(kali㉿kali)-[~]  
$ ./a.out
```

===== MATRIX DISTRIBUTIVE PROPERTY =====

1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit

Enter your choice: 1

Enter m, n and p: 2 2 2

Enter elements of Matrix A (2 x 2):

1 2 3 4

Enter elements of Matrix B (2 x 2):

5 6 7 8

Enter elements of Matrix C (2 x 2):

1 1 1 1

Left Hand Side: $A(B + C)$

22 25

50 57

Right Hand Side: $AB + AC$

22 25

50 57

Left distributive property is VERIFIED.

Test Case 2:

```
===== MATRIX DISTRIBUTIVE PROPERTY =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 2 2

Enter elements of Matrix A (2 x 2):
1 2 3 4

Enter elements of Matrix B (2 x 2):
5 6 7 8

Enter elements of Matrix C (2 x 2):
1 1 1 1

Left Hand Side: (A + B)C
14      14
22      22

Right Hand Side: AC + BC
14      14
22      22

Right distributive property is VERIFIED.
```

Test Case 3:

```

===== MATRIX DISTRIBUTIVE PROPERTY =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2 3 2

Enter elements of Matrix A (2 x 3):
1 2 3
4 5 6

Enter elements of Matrix B (3 x 2):
1 2
3 4
5 6

Enter elements of Matrix C (3 x 2):
6 5
4 3
1 2

Left Hand Side: A(B + C)
39      45
99      111

Right Hand Side: AB + AC
39      45
99      111

Left distributive property is VERIFIED.

```

Test Case 4:

```

===== MATRIX DISTRIBUTIVE PROPERTY =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 3 2

Enter elements of Matrix A (2 x 3):
1 2 3
4 5 6

Enter elements of Matrix B (2 x 3):
6 5 4
3 2 1

Enter elements of Matrix C (3 x 2):
1 2
3 4
5 6

Left Hand Side: (A + B)C
63      84
63      84

Right Hand Side: AC + BC
63      84
63      84

Right distributive property is VERIFIED.

```

Test Case 5:

```

===== MATRIX DISTRIBUTIVE PROPERTY =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 3

Program terminated.

```

Test Case 6:

```
(kali㉿kali)-[~]  
$ ./a.out
```

```
===== MATRIX DISTRIBUTIVE PROPERTY =====
```

1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit

Enter your choice: 4

Invalid choice! Please enter 1, 2 or 3.